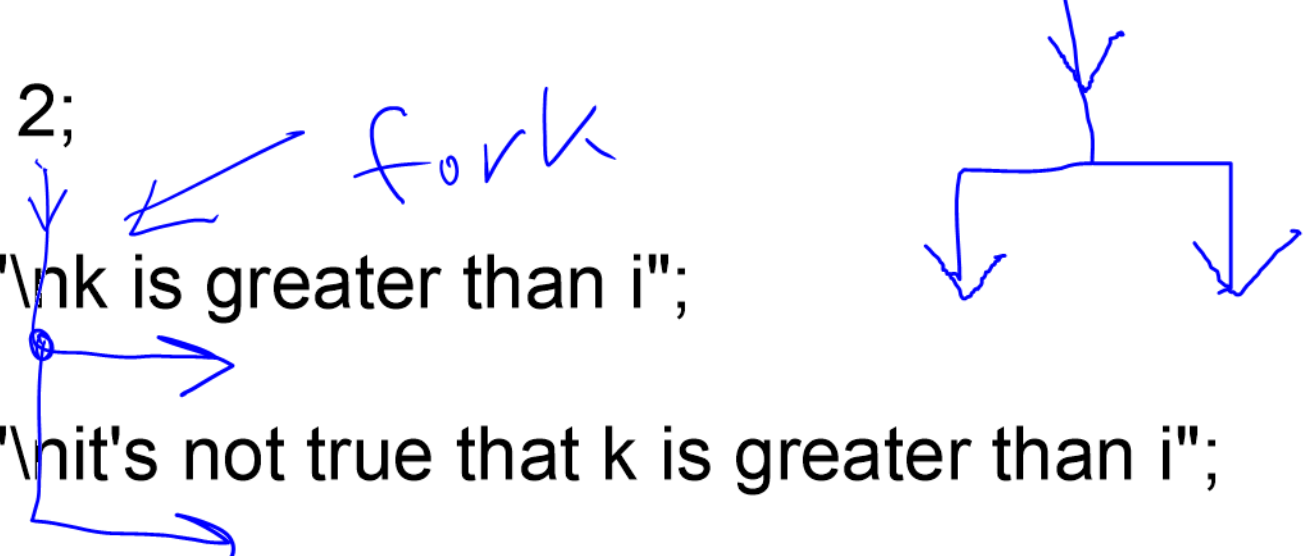


if - else statement

if the test condition in () is true then the 1st codeblock is executed, otherwise the 2nd codeblock is executed, eg.

```
int i = 1; k = 2;  
if( k > i ) {  
    cout << "\nk is greater than i";  
} else {  
    cout << "\nit's not true that k is greater than i";  
}
```



the else statement is like a default option that occurs when the if statement is not true

an if-else statement is like a "fork in the road" since the program must follow one of the two code paths / blocks

if - else ladders

It's possible to have one if statement inside of another (ie "nested"), for example:

```
if ( i == 1 ) {  
    cout << "\ni == 1";  
} else { ←  
    if ( i == 2 ) {  
        cout << "\ni == 2";  
    } else { ←  
        if ( i == 3 ) {  
            cout << "\ni == 3";  
        } else { ←  
            cout << "\ni is not equal to 1, 2, 3 -- none of the above";  
        } ←  
    } ←  
} ←
```

For this example, only one if statement is executed depending on the value of i. The last else occurs if none of the if statements is executed -- only one of the if statements or final else occurs.

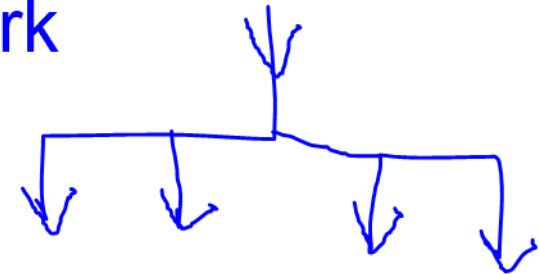
The example above can be written as:

```
if ( i == 1 ) {  
    cout << "\ni == 1";  
} else if ( i == 2 ) {  
    cout << "\ni == 2";  
} else if ( i == 3 ) {  
    cout << "\ni == 3";  
} else {  
    cout << "\ni is not equal to 1, 2, 3 -- none of the above";  
}
```

ladder steps

control statement
"lego"

generalized
fork



This statement is known as an if-else ladder -- it's very useful for developing programs that make decisions

An if-else ladder is analgous to a fork in the road with more than two paths -- only one of the code paths / blocks is executed

The ladder is executed one if statement at a time from the top down (ie top of ladder to the bottom) until one condition is true

The last else statement is the default condition -- it can be removed but it's not recommended since it's no longer a true fork and the default often checks for errors

The switch statement in C++ is similar to if-else ladders, but it is less general since it can't use variables in the test conditions -- I don't recommend / teach switch statements

Loop Statements

A loop is used to execute a part of the program repeatedly

The for loop is the most common type of loop, eg

0. initialize index variable 1. test condition 4)
3. index update 6)
i=0,1,2,3,4

```
for( i = 0 ; i < 5 ; i++ ) {  
    cout << "\ni = " << i;  
}
```

2. execute codeblock 5)

ⁱ⁼⁵
the loop repeats (steps 1 to 3) until the test condition is false

note that the {} is not needed for single line code blocks, eg

```
for( i = 0; i < 5; i++ ) cout << "\ni = " << i;
```

generalized increment and other types of expressions can be used in for loops, eg

```
int i, k; double x;
```

```
for( i = 0; i <= 10; i += 2 ) cout << "\n" << i;
```

```
for( x = 1.0; x <= 1.0e5; x *= 10.0 ) cout << "\n" << x;
```

```
for( k = 0; k < 1000; k = k*k + 1 ) cout << "\n" << k;
```

note the exit value of a for loop is the final incremented index that no longer satisfies the loop condition, eg

```
cout << "\ni = " << i;
```

```
cout << "\nx = " << x;
```

it's possible to put a for loop inside a for loop (ie a nested loop)

this is very useful for 2D arrays, matrices, etc. eg,

```
double A[3][3];
```

```
for(i=0;i<3;i++) { // outer loop (i = row)
    // for each i this inner loop goes over all j
    for(j=0;j<3;j++) { // inner loop (j = col)
        cout << "\ni = " << i << " , j = " << j;
        A[i][j] = 1.0 + i + j; // initialize A
    }
}
```

therefore, this example sets a 2D array one row at a time